Overview

Task: Using the provided protein expression matrix *"proteomics_data.txt"*, construct a protein–protein interaction (PPI) network from the data, detect modules/communities within that network, and use this protein network to mind the differences cross the 118 patient sample.

I have organised my work into four sequential Jupyter notebooks, each building on the output of the previous one:

Notebook	What I did in it
Notebook 1: Data Preprocessing	Loaded the data, filtered low coverage proteins, imputed missing values, and built the CORUM gold standard PPI list
Notebook 2: PPI Prediction	Computed pairwise correlations, benchmarked them against CORUM, and produced a ranked PPI table
Notebook 3: Network & Modules	Built the PPI graph, characterised its topology, and ran community detection to find protein modules
Notebook 4: Functional Annotation	Computed module eigengenes, clustered patients, ran GO enrichment, and interpreted the biology

Stage 1: Preprocessing the Data and Predicting PPIs

Raw input matrix

The expression matrix had 12,241 proteins × 118 patients, with values representing log2 ratios from mass spectrometry. The first thing I noticed was the missing data: 244,858 NaN entries. This is completely expected in proteomics, and also it was mentioned in the task description that NA in the proteomics dataset meant that the protein has not been quantified in the sample, which could mean MNAR, if the abundance was below the detection limit

Figure 1 shows the distribution of all measured expression values, sampled from the full dataset. The values are centred near zero on the log2-ratio scale. This confirms the data looks good and normally distributed around the reference level.

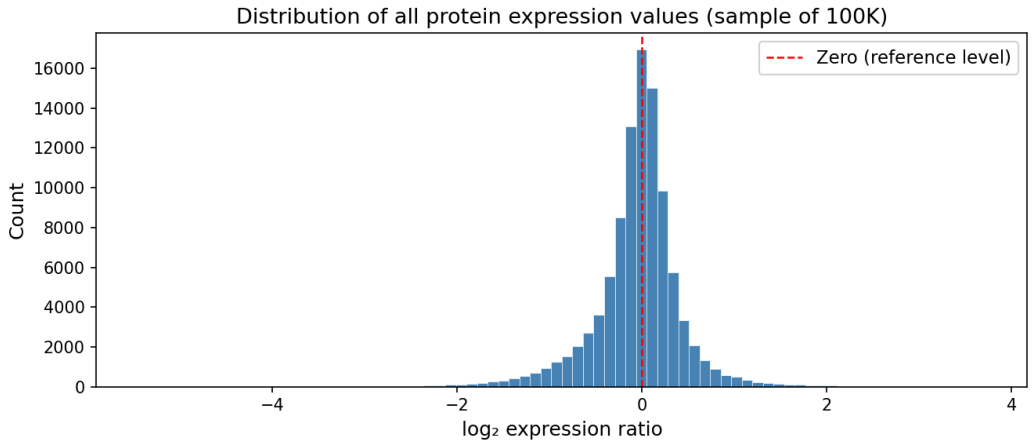

Figure 1. Distribution of protein expression values across all patients (sample of 100K measurements). Values are log2 ratios, centred near zero.

Filtering: deciding which proteins to keep

I have removed any protein detected in fewer than 50% of patients. I did this because if a protein is only seen in 10 or 20 out of 118 samples, any correlation I compute for it will be based on a small number of data points and won't be reliable. I believe that the 50% threshold is a commonly used cutoff in the field.

Before filtering	After filtering	Removed
12,241 proteins	10,164 proteins	2,077 proteins

Imputation: filling in the blanks

For proteins that passed the filter but still had some missing values, I used a left tail imputation strategy. The idea behind this is that when a protein is not detected in a sample, it's almost certainly because its abundance was below the instrument's detection limit, and not completely absent. So instead of filling the NaNs with mean or median, I drew replacement values from a distribution shifted downward from the protein's observed mean:

- Compute the mean and standard deviation of the protein's detected values
- Shift the distribution down by 1.8 standard deviations
- Shrink the spread to 0.3 × the original standard deviation
- Sample random values from this low end distribution to fill the gaps

I believe it does a reasonable job of preserving the true correlation structure.

Building the CORUM gold standard

To evaluate how good my PPI predictions are, I needed a set of known interactions to test against. I used the CORUM database, which has experimentally confirmed protein complexes. Proteins in the same complex must interact, so all pairwise combinations within a complex count as known PPIs. Parsing the CORUM file gave me 39,430 positive PPI pairs to use as ground truth.

Predicting PPIs from co-expression

The core biological assumption is that proteins which interact tend to be co-regulated. If one goes up, the other goes up too, across patients. I quantified this by computing pairwise correlations across the 118-patient expression profiles, and we get a 10,164 × 10,164 correlation matrices for both Spearman and Pearson.

Figure 2 shows that CORUM protein pairs, which we know physically interact, cluster at much higher Spearman rho values than random pairs.

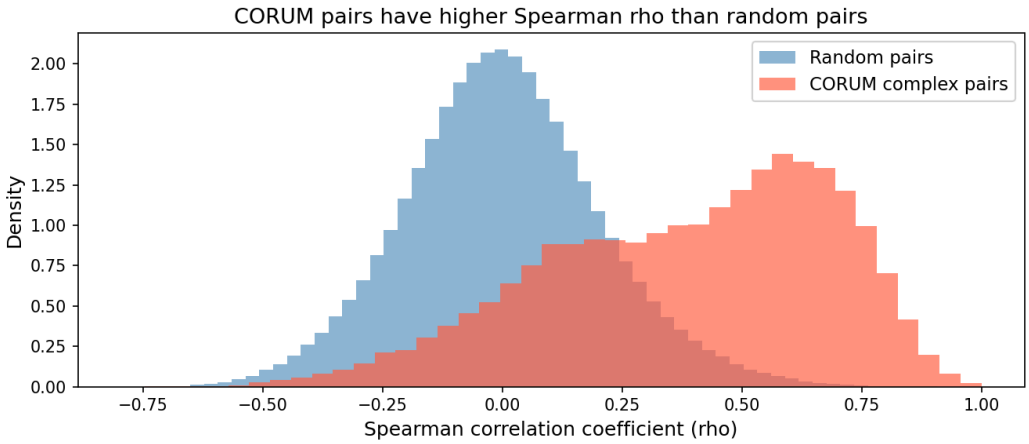

Figure 2. Distribution of Spearman rho scores for CORUM complex pairs (orange) vs. random protein pairs (blue). Known interactors score systematically higher, validating co-expression as a PPI predictor.

Benchmarking

I compared Spearman and Pearson against 500,000 randomly sampled non-interacting pairs, using AUROC (ranking ability) and AUPRC (precision-recall, which is more informative when positives are rare):

Method	AUROC	AUPRC	Lift over random
Spearman rho	0.848	0.567	7.8×
Pearson r	0.846	0.559	7.6×
Random baseline	0.500	0.073	1.0×

Figure 3 shows the ROC and Precision-Recall curves. Both methods are far better than random , nearly 8× lift in precision-recall. I chose Spearman for the final analysis because it's rank-based and more robust to the non-normal distribution of imputed values and biological outliers.

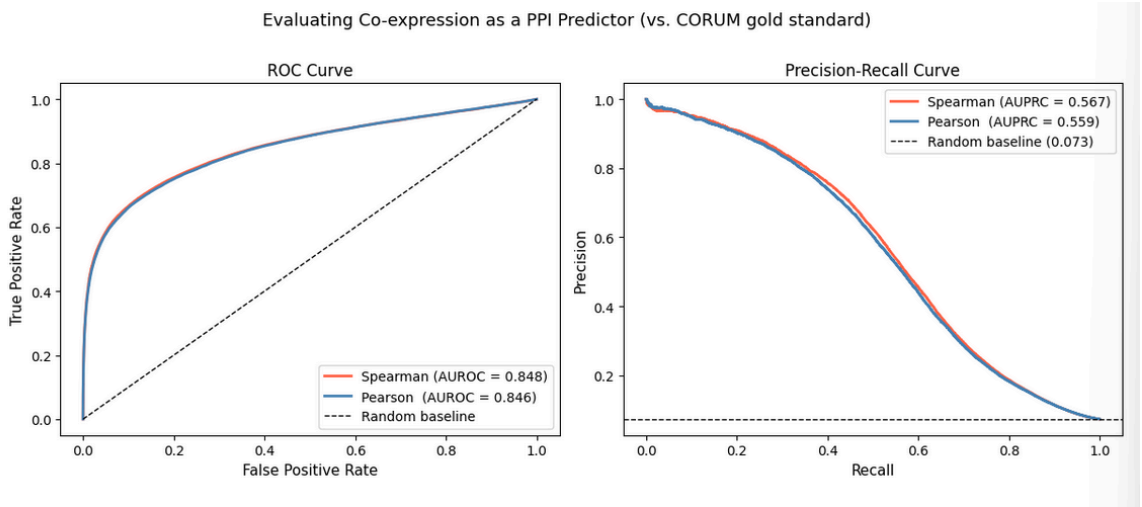

Figure 3. ROC curves (left) and Precision-Recall curves (right) for Spearman and Pearson correlation as PPI predictors, benchmarked against CORUM. Both methods outperform the random baseline (dashed line).

Generating the ranked PPI table

I filtered the full correlation matrix to retain only pairs with rho ≥ 0.8 - giving 8,987 predicted protein pairs, saved as `outputs/network_edges.tsv`. The top-ranked pair was FGB - FGG (rho = 0.994), both fibrinogen subunits - a known complex, which is a reassuring sanity check.

Stage 2: Building the Network and Finding Modules

Constructing the graph

I loaded the edge list into NetworkX and built an undirected, weighted graph where each node is a protein and each edge is a predicted interaction weighted by Spearman rho:

Property	Value
Nodes (proteins)	1,658
Edges (interactions)	8,987
Network density	0.0065 (very sparse)
Average degree	10.84 connections per protein
Connected components	184 separate sub-graphs
Giant component	449 proteins (27.1% of all nodes)

The sparsity and modularity are actually expected features of biological networks since real interactomes have a similar scale-free structure. The giant connected component is where most of the biologically interesting proteins cluster, so I focused my community detection there.

Finding the Hub proteins

I ranked all proteins by their number of connections. The top ten were:

Rank	Protein	Connections
1	DDX17	96
2	SF3B2	89
3	TOP2A	87
4	KIF2C	85
5	U2AF2	85
6	KIF11	84
7	KIF22	83
8	TTK	83
9	CENPF	80
10	HMMR	80

Community detection

I applied community detection to the giant component. I tested both Louvain and Leiden algorithms and chose Leiden for the final result because it guarantees well-connected communities, since we know Louvain can produce internally disconnected ones, and it gives more reproducible results between runs.

Leiden found 14 modules with a modularity score of 0.4944..

Module size distribution

Figure 4 shows how proteins are distributed across the 14 modules. The three largest modules (0, 1, 2) each contain around 100 proteins and seem to represent the major functional complexes of the cell. The remaining modules are smaller and seem to be more specific functional complexes.

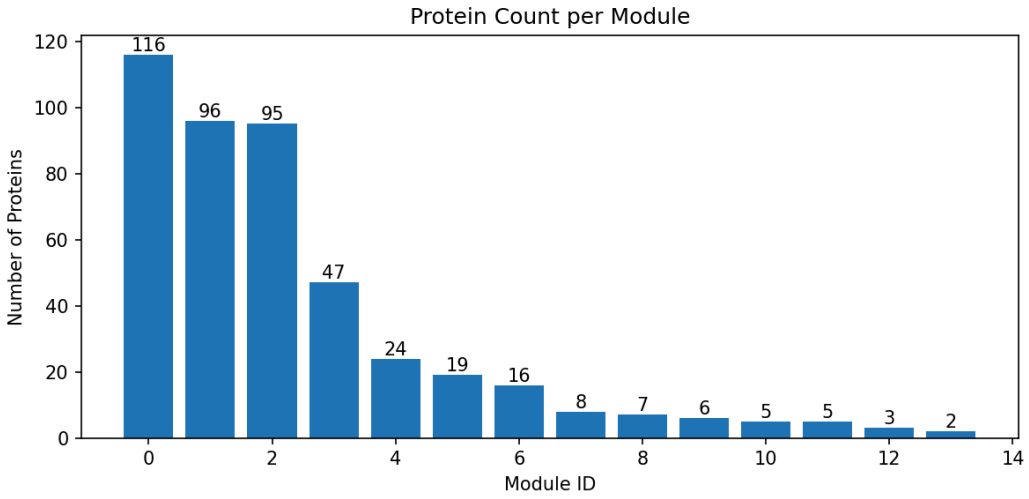

Figure 4. Number of proteins per Leiden module. Modules are sorted by size. The three largest modules (0, 1, 2) are roughly equal in size at 100 proteins each.

Module	Size	Biological theme
0	116	Nucleic acid biosynthesis
1	96	Ribosome biogenesis
2	95	mRNA processing
3	47	Regulation of endogenous retroelements
4	24	rRNA processing in the nucleus and cytosol
5	19	Processing of capped intron-containing pre-mRNA
6	16	RNA polymerase complex
7	8	RNA binding
8	7	mRNA binding
9	6	mRNA 3'-end processing
10	5	Transcription initiation by RNA Pol II
11	5	mRNA Splicing - Major Pathway
12	3	Nucleocytoplasmic transport
13	2	Splicing factor binding

Stage 3 - Functional Annotation and Patient Interpretation

My plan for annotation

I organised the functional annotation into four steps:

1. **Module eigengenes** - for each module, I computed the mean expression of all its member proteins across every patient. This gives a single summary value per patient per module, making the data manageable for clustering.
2. **Patient clustering** - I ran K-Means on the 118 × 13 eigengene matrix to group patients into biologically similar clusters. I used the silhouette score to pick the optimal number of clusters.
3. **GO enrichment** - for each module's gene list, I ran GO term enrichment using g:Profiler to find which biological processes are statistically overrepresented.
4. **Interpretation** - I linked the patient clusters to module activity, and module activity to the enriched biology, to build a coherent picture.

Visualising module expression across patients

Before clustering, I plotted the eigengene heatmap - module activity for all 118 patients (Figure 5). Each row is a patient, each column a module, and the colour indicates whether that module's proteins are on average up (red) or down (blue) in that patient. Even without formal clustering, we can already see two broad patient groups forming, a group with globally higher module activity (top of the heatmap) and a group with lower activity (bottom). The modules don't seem to behave the same way, which means that our clustering is genuinely informative.

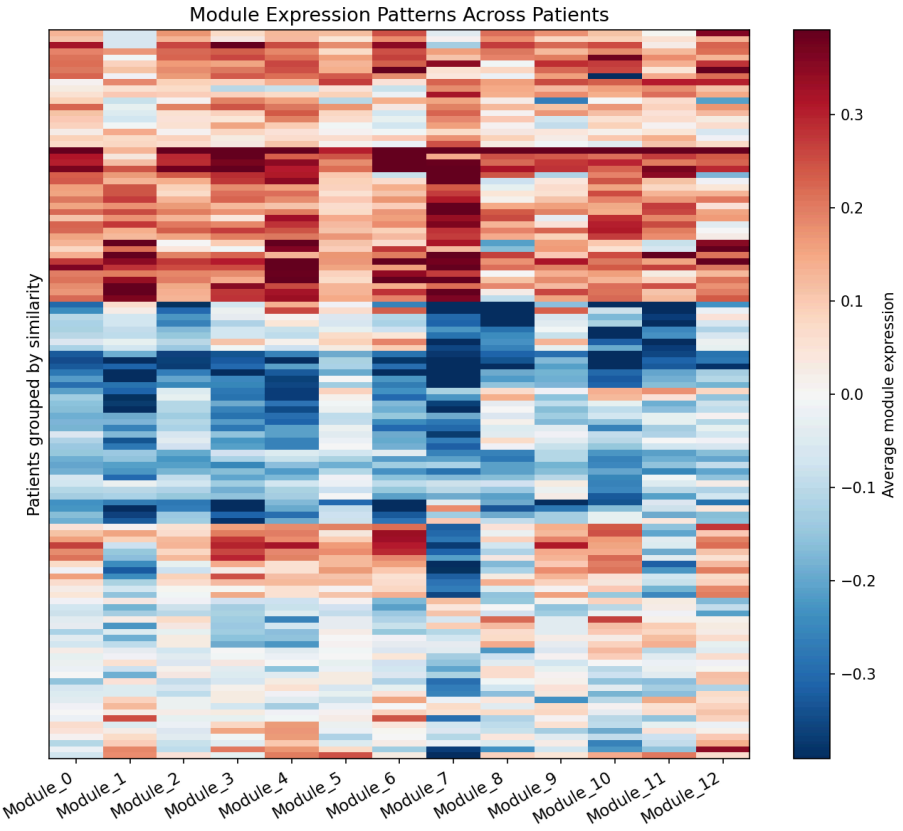

Figure 5. Heatmap of module eigengene expression across all 118 patients (patients grouped by similarity). Red indicates above average module activity, blue indicates below average.

Choosing the number of patient clusters

To decide how many patient clusters to use, I ran K-Means for K = 2 through 6 and calculated the silhouette score for each. Figure 6 shows the result: K = 2 gives us the highest silhouette score (0.374), and the score drops off consistently as K increases. This strongly supports a two-cluster solution.

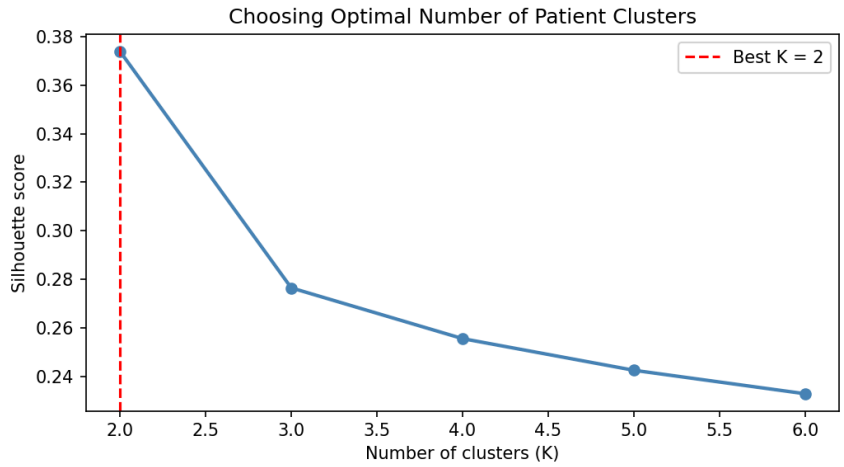

Figure 6. Silhouette scores for K-Means clustering of 118 patients on the module eigengene matrix. K = 2 (dashed red line) achieves the highest score, indicating it is the most internally consistent partition.

Patient clustering result

The K-Means clustering split the 118 patients into two groups:

Cluster	N patients	What distinguishes them
Cluster 0	53	Higher activity in Module 3 (chromatin remodeling) and Module 4 (rRNA processing)
Cluster 1	65	Higher activity in Module 5 (pre-mRNA splicing/capping) and Module 12 (nucleocytoplasmic transport); notably lower ribosome biogenesis (Module 1)

Figure 7 shows the averaged module activity per cluster. Cluster 0 (top row) is uniformly red across all modules - these patients have globally elevated expression of all the detected protein complexes. Cluster 1 (bottom row) is consistently blue. Module 4 (rRNA processing) is the darkest red in Cluster 0 and Module 7 (RNA binding) is the most suppressed in Cluster 1, both of which point to real biological differences between the groups.

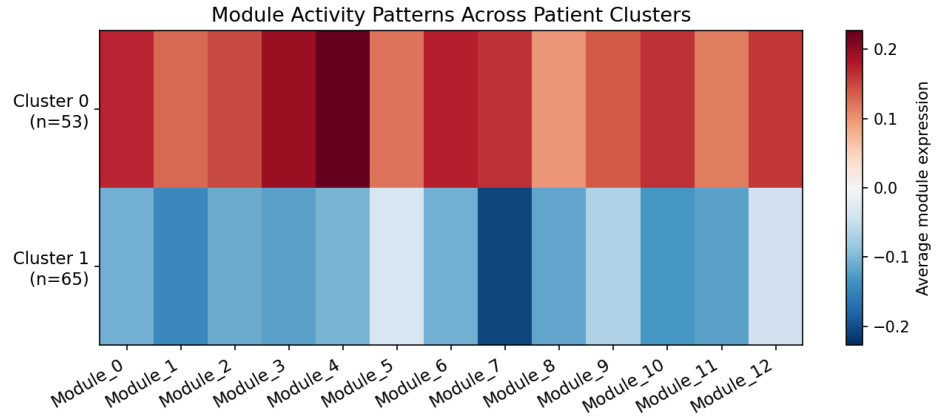

Figure 7. Average module eigengene activity per patient cluster. Cluster 0 (n=53) shows globally elevated module activity (red), while Cluster 1 (n=65) shows reduced activity (blue).

Worked example 1: Module 0 - RNA processing hub

116 proteins | Top hubs: DDX17, U2AF2, ACTL6A, NCL, SNRNP70

The GO enrichment for this module was extremely strong:

GO term	p-value
Nucleic acid biosynthetic process	6.134772e-32
RNA metabolic process	6.134772e-32
mRNA metabolic process	6.134772e-32
mRNA processing	6.134772e-32
RNA biosynthetic process	6.134772e-32

This module is enriched with RNA-binding proteins, helicases, and splicing factors. Clinically, this is highly relevant because mutations in splicing factors are among the recurrent oncogenic drivers reported in AML. The emergence of this pathway as one of the largest and most densely connected modules in the network suggests that the approach is capturing biologically meaningful structure consistent with known AML biology.

DDX17 was the most connected protein in the network, with 96 interactions. It is known to play a role in important RNA-related processes such as RNA processing and gene regulation. Since it appears at the center of many protein interactions, it could be an interesting protein for further investigation as a possible biomarker or therapeutic target.

In the patient data, Cluster 1 (n = 65) had lower overall activity in this module compared to Cluster 0, hinting at different dependencies on RNA biosynthesis, possibly reflecting different underlying driver mutations.

Worked example 2: Module 3 - Epigenetic regulation

47 proteins | Top hubs: CHD4, SMARCA2, KDM1A, SMARCC2, SMARCE1, SMARCD2, SMARCB1

Many of the proteins in this group are known to be involved in epigenetic regulation, which controls how genes are turned on or off without changing the DNA sequence itself.

- **SWI/SNF chromatin remodeling proteins**: SMARCA2, SMARCC2, SMARCE1, SMARCD2, SMARCB1: co-regulated as a group
- **NuRD complex proteins**: CHD4 and MBD3
- **Histone-modifying enzymes**: KDM1A

What I found interesting is that many of these proteins appeared together in the same co-expression module. This may suggest that they are functioning in a coordinated way in these patients and could represent an important epigenetic regulatory program.

Module 3 is one of the most active module in Cluster 0 (n = 53 patients). Although clinical validation would still be needed, observations like this could help generate hypotheses about possible biomarkers or treatment responses in the future.

Key Results

Step	Result
Data after filtering	10,164 proteins (from 12,241), with missing values imputed
PPI prediction quality	Spearman rho: AUROC = 0.848, AUPRC = 0.567, 7.8× lift over random
Network size	1,658 proteins, 8,987 edges, 184 connected components
Modules detected	14 Leiden communities (modularity = 0.49)
Patient clusters	2 clusters (n=53 and n=65) with distinct module activity profiles

Key biology found	RNA processing (Modules 0–2) and epigenetic regulation (Module 3) dominate

Limitations

1. Co-expression is a proxy, not direct evidence. High correlation between two proteins tells me they're co-regulated, but not that they physically touch. Many of the 8,987 predicted edges will be false positives

2. The imputation affects the correlations. I used Left tail imputation over the median approach, but it does introduce assumptions.A different imputation method (KNN, probabilistic PCA) might shift some correlation values and change which pairs cross the rho >= 0.8 threshold.

3. Choosing rho >= 0.7 threshold. I picked 0.8 after manually going over multiple iterations, because it gave a reasonable network size and a scale free network, but I didn't systematically scan thresholds. A sensitivity analysis would strengthen the choice.

4. I only clustered the giant component. The Leiden algorithm ran on the 449-protein giant connected component. The other ~1,200 networked proteins in smaller components don't have module assignments.

5. No clinical metadata available. Without AML subtype, mutation status, cytogenetics, or survival data, the patient cluster interpretation stays at the level of 'interesting proteomic differences'. Linking this to outcomes is the obvious and critical next step.

6. GO enrichment favours well-studied biology. Processes with lots of literature get annotated better in GO. Newer or less-studied aspects of AML biology might be present in the modules but not captured by GO terms.

What I would do given more time

1. Integrate more external PPI databases (STRING, BioGRID). Combining co-expression evidence with experimentally confirmed interactions would reduce false positives and give more confidence in the high-priority edges.
2. Scan multiple correlation thresholds. Compute AUPRC on held-out CORUM pairs at thresholds from 0.5 to 0.9 and pick the one that maximises precision-recall, rather than using a manually chosen value of 0.8.
3. Trying different correlation methods and also considering clinical information.
4. I would extend this workflow using Retrieval-Augmented Generation (RAG), biomedical knowledge graphs, and agentic AI to automate biological interpretation of detected protein modules. The system could retrieve evidence from literature and pathway databases (e.g., PubMed, Reactome, STRING) and use specialized AI agents for pathway analysis, drug-target discovery, and hypothesis generation.
5. Extend module detection beyond the giant component. 184 connected components exist. The smaller ones might contain biologically interesting isolated complexes that I've currently missed.